

Azure DevOps Pipelines

Complete Developer Interview Notes

Enterprise-oriented CI/CD with Azure DevOps, .NET 8, GitHub, SonarCloud, templates, environments and deployment

GitHub → PR Validation → main → Build → Artifact → Dev → QA → Production

■■■ Tests + Coverage + SonarCloud ■■■

Learning focus: Understand the architecture and reasoning first; learn YAML as the implementation of those concepts. The depth is intentionally aimed at a .NET developer interview rather than a DevOps-administrator interview.

Contents

1. Course scope and mental model
2. CI/CD fundamentals
3. Azure DevOps architecture
4. Agents, jobs, stages and steps
5. Artifacts and Build Once, Deploy Many
6. Tasks vs scripts
7. Variables, variable groups and secrets
8. Deployment jobs and Azure environments
9. Service connections
10. Conditions and dependencies
11. Triggers and PR validation
12. Release branches and feature flags
13. Deployment strategies
14. Enterprise pipeline architecture
15. Templates and parameters
16. Parameters vs variables vs variable groups
17. Complete build and deployment YAML
18. SonarCloud and code coverage
19. Complete PR validation YAML
20. GitHub + Azure DevOps integration
21. Pipeline troubleshooting
22. Interview questions and answers
23. Final cheat sheet

1. Course Scope and Mental Model

This course uses a single sample application, **EmployeeManagementAPI**, as the running example. The goal is to understand how a real enterprise pipeline is structured, not merely how to write YAML.

Developer-level target

- Know why CI/CD is structured into build, test, artifact and deployment stages.
- Understand what an agent, job, stage, step, artifact and environment does.
- Be able to read and explain Azure Pipelines YAML.
- Understand PR validation, SonarCloud, code coverage and quality gates.
- Understand templates, parameters, variables and variable groups.
- Explain approvals, service connections and deployment strategies.
- Troubleshoot common failures logically.

What we intentionally do not go deep into

- Designing large self-hosted-agent fleets.
- Kubernetes administration and advanced container orchestration.
- Complex multi-cloud release orchestration.
- Writing custom Azure DevOps extensions.
- DevOps-administrator-level identity and governance details.

Core mental model

```
Source code → Build → Test → Package → Artifact → Deploy
```

■

■■■ Quality: Sonar + Coverage

The most important principle is **Build Once, Deploy Many**: produce one tested artifact and promote that same artifact through environments instead of rebuilding different binaries for Dev, QA and Production.

2. CI/CD Fundamentals

Continuous Integration (CI)

CI means integrating code changes frequently and automatically validating them. A typical CI pipeline restores dependencies, builds the solution, runs tests, performs quality checks and optionally creates a deployable artifact.

Continuous Delivery / Deployment

Continuous Delivery means the software is always in a deployable state. Continuous Deployment goes one step further and automatically deploys when the required conditions are satisfied.

```
Developer → Feature branch → Pull Request
```

↓

```
Build / Test / Sonar
```

↓

```
Merge
```

```
↓
main pipeline
↓
Artifact
↓
Dev → QA → Prod
```

Why CI and CD are separated conceptually

PR validation answers: **Is this change safe to merge?** The main pipeline answers: **Can this exact source state produce and promote an official artifact?**

3. Azure DevOps Architecture

```
Azure DevOps
■■■ Repos → source control (our actual source is GitHub)
■■■ Pipelines → CI/CD execution
■■■ Artifacts → packages/build outputs
■■■ Environments → deployment targets + history + checks
■■■ Service Connections → authenticated access to external resources
```

Pipeline hierarchy

```
Pipeline
■■■ Stage(s)
■■■ Job(s)
■■■ Step(s)
■■■ Task
■■■ Script
■■■ Checkout
```

Stage

A stage is a logical phase such as Build, Development, QA or Production. Stages are useful for sequencing and approvals.

Job

A job is a collection of steps that execute together on an agent. Different jobs normally execute on separate agents, so files should not be assumed to exist across jobs unless passed through artifacts or other mechanisms.

Step

A step is an individual action: a task, script or checkout operation.

4. Agents, Jobs, Stages and Steps

Agent

An agent is the machine that executes pipeline work. For learning, Microsoft-hosted agents are preferred; they provide a fresh build environment and remove the need to maintain a server.

Why fresh agents matter

A deployment job should not assume it has the build workspace. This is why the deployment stage downloads the published artifact.

```
Build agent
checkout → restore → build → test → publish artifact
↓
Artifact: drop
↓
Deployment agent
download artifact → deploy
```

Interview point

If two jobs run on different agents, files created in Job 1 are not automatically available in Job 2. Use artifacts when the output must cross job/stage boundaries.

5. Artifacts and Build Once, Deploy Many

Artifact

An artifact is a packaged output produced by a pipeline and made available to later stages or pipelines.

```
Build
↓
EmployeeManagementAPI.zip
↓
Artifact: drop
↓
Dev → QA → Production
```

Why not rebuild per environment?

- A rebuild can produce a different binary.
- You lose confidence that QA tested exactly what Production receives.
- It makes rollback and traceability harder.

Enterprise principle: Build once, test the artifact, and promote the same artifact.

6. Tasks vs Scripts

Script

```
- script: dotnet build
```

Scripts give maximum command-line flexibility.

Built-in task

```
- task: DotNetCoreCLI@2
  displayName: 'Build Solution'
  inputs:
    command: 'build'
    projects: '**/*.sln'
```

Built-in tasks provide Azure Pipelines integration, structured inputs and clearer logs. For common operations, prefer built-in tasks; use scripts when the operation is custom or awkward to express through a task.

Common .NET tasks

- UseDotNet@2 – install/select a .NET SDK.
- DotNetCoreCLI@2 – restore, build, test, publish and package .NET applications.
- PublishTestResults@2 – publish test result files.
- PublishCodeCoverageResults@2 – publish coverage reports.
- AzureWebApp@1 – deploy to Azure App Service.

7. Variables, Variable Groups and Secrets

Variables

Variables hold runtime configuration or values used by tasks.

```
variables:
- name: BuildConfiguration
  value: Release
```

Variable Group

A variable group stores shared configuration outside the YAML. It is useful when several pipelines or environments need centrally managed values.

```
variables:
- group: Common
```

Environment-specific configuration

```
Common variable group → shared values
Dev variable group → Dev App Service name/settings
QA variable group → QA App Service name/settings
Prod variable group → Prod App Service name/settings
```

Secrets

A DbConnection string or other secret is not needed to compile C# merely because it is a connection string. It becomes relevant when tests, integration tests, migrations, deployment scripts or application configuration require it. The application normally receives runtime configuration when it runs, not because the compiler needs a database connection.

Key principle

Separate configuration from implementation. Never hard-code passwords, tokens or production connection strings in YAML or source code.

8. Deployment Jobs and Azure Environments

Deployment job

```
jobs:
```

```

- deployment: DeployWebApi
  environment: QA
  strategy:
    runOnce:
      deploy:
        steps:
          - download: current
            artifact: drop

```

A deployment job is designed for deployment scenarios and integrates with Azure DevOps Environments.

Environment

An environment is a logical representation of a deployment target such as Development, QA or Production. It can provide deployment history, approvals and checks.

```

Pipeline
↓
environment: Production
↓
Azure DevOps checks/approval
↓
deployment job runs

```

Important distinction

The YAML references an environment; approvals are configured on the environment in Azure DevOps. You do not normally hard-code a human approval into the deployment task.

9. Service Connections

A service connection provides Azure DevOps with an authenticated identity for accessing an external service, such as an Azure subscription.

Do we need one connection per environment?

No. Three environments do not automatically require three service connections. If one service connection has sufficient permissions to the Dev, QA and Production resources, the same connection can be reused.

```

Azure DevOps
■
■■■ Azure Service Connection
■■■ EmployeeApi-Dev
■■■ EmployeeApi-QA
■■■ EmployeeApi-Prod

```

In a stricter enterprise setup, separate identities/connections can reduce blast radius and enforce least privilege. That is a security design choice, not a YAML requirement.

Developer interview answer

A service connection answers **who is allowed to access the target resource?** An environment answers **which deployment target and checks are associated with this deployment?**

10. Conditions and Dependencies

Common conditions

```
condition: succeeded()
condition: succeededOrFailed()
condition: failed()
condition: always()
```

- **succeeded()** – run only when the preceding required execution succeeded.
- **succeededOrFailed()** – run after success or failure; useful for publishing test diagnostics even when tests fail.
- **failed()** – run when a preceding operation failed.
- **always()** – run regardless of the previous result, including cancellation scenarios where applicable.

Why publish test results after a test failure?

```
Build ✓
Tests ✗
Publish results ✓ ← condition: succeededOrFailed()
Overall pipeline ✗
```

The condition does not ignore the failure. It allows the reporting task to run so the developer can see the failed tests.

dependsOn

```
dependsOn: Deploy_Development
```

Dependencies establish stage/job ordering. For our deployment flow, QA depends on Development and Production depends on QA.

11. Triggers and PR Validation

Common trigger types

- CI/push trigger – run when code is pushed to selected branches.
- PR trigger – validate a pull request.
- Manual trigger – run on demand.
- Scheduled trigger – run at a defined time.
- Pipeline completion – start another pipeline after a pipeline finishes.

Our PR validation trigger

```
trigger: none

pr:
  branches:
    include:
      - master
```

With GitHub as the repository, a pull request targeting the configured branch can cause Azure Pipelines to start the PR validation pipeline. The repository sends the PR event; Azure DevOps evaluates the pipeline trigger.

Main CI/CD trigger

```
trigger:
  branches:
    include:
      - master
```

When a PR is merged, the resulting commit on master can trigger the main CI/CD pipeline.

12. Release Branches and Feature Flags

Why a release branch?

A release branch is a snapshot/freeze point used to stabilize a planned release while main may continue evolving.

```
main ■■ A ■■ B ■■ C ■■ D ■■ E
■
■■■ release/1.0
```

The common problem

Feature ABC is intended for release, but XYZ is merged into main before the release branch is cut. If the release branch is cut after XYZ, XYZ is included too.

Ways to reduce this risk

- Create the release branch at the appropriate freeze point.
- Use feature flags so incomplete features can exist without being exposed.
- Coordinate release scope and code freeze.
- Avoid merging unfinished release features into the branch that will become the release snapshot.

Interview point

A release branch is not a magic filter. It is a branch created from a particular commit, so it contains whatever that commit already contains.

13. Deployment Strategies

RunOnce

Deploy the application once to the target environment. This is sufficient for our sample App Service deployment.

Rolling

Update instances gradually, reducing the number of instances changed at one time.

Blue-Green

```
Blue = current production
Green = new version
Deploy Green → validate → switch traffic → Blue becomes standby
```

Canary

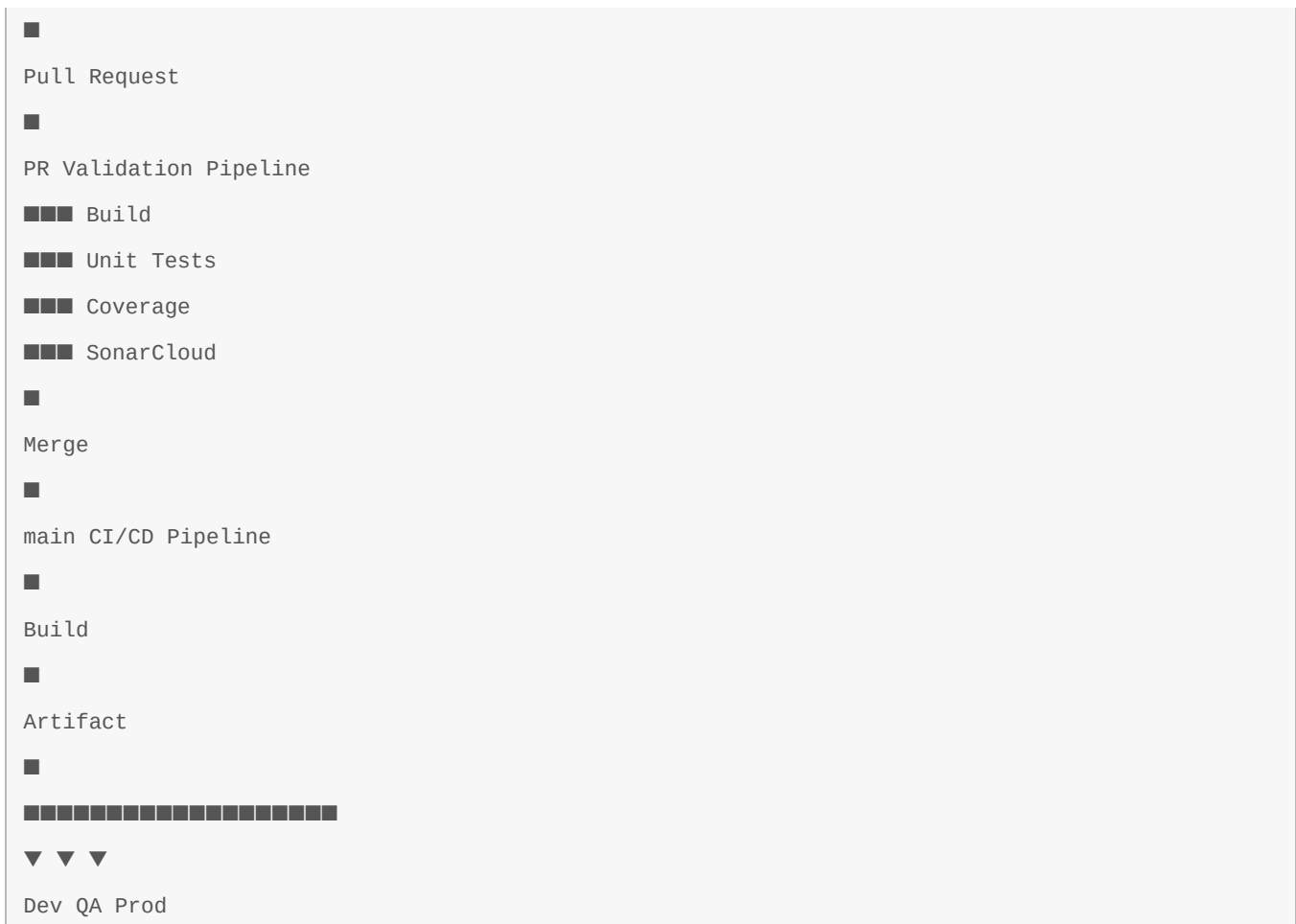
Release the new version to a small percentage of traffic/users first, observe behavior, then expand.

Environment vs strategy

Environment answers **where** you deploy. Strategy answers **how** you roll out the deployment.

14. Enterprise Pipeline Architecture

```
GitHub
```



Separation of responsibilities

- PR pipeline validates proposed changes.
- Main pipeline creates the official deployable artifact.
- Templates contain reusable pipeline logic.
- Variable groups contain configuration.
- Environments contain deployment targets and checks.
- Service connections provide authenticated access.

15. Templates and Parameters

Why templates?

Without templates, many pipelines duplicate the same YAML. Templates centralize build/deploy logic and make changes consistent.

```
azure-pipelines.yml
■
■■■ templates/build.yml
■
■■■ templates/deploy.yml
```

Parameters

```
- template: templates/deploy.yml
parameters:
  environmentName: QA
  variableGroup: QA
  dependsOn: Deploy_Development
```

Parameters shape the template when the pipeline is expanded. They are useful for values such as environment names and template behavior.

C# analogy

A YAML template is similar to a reusable method/class template, while parameters are similar to method arguments.

16. Parameters vs Variables vs Variable Groups

Concept	When used	Typical example
Parameters	Template/pipeline expansion	environmentName: QA
Variables	Runtime pipeline values	BuildConfiguration=Release
Variable Groups	Shared external configuration	Common / Dev / QA / Prod

Lifecycle mental model

```
Parameters → pipeline/template expansion
Variables → runtime execution
Variable Groups → shared configuration/secrets
```

17. Complete Build and Deployment YAML

The following is the consolidated, interview-ready implementation built during the course. It is intentionally kept practical rather than administrator-heavy.

Repository structure

```
EmployeeManagementAPI/
  src/
  tests/
  templates/
    build.yml
    deploy.yml
  variables/
    common.yml
    dev.yml
    qa.yml
    prod.yml
  pr-validation.yml
  azure-pipelines.yml
```

Main pipeline: azure-pipelines.yml

```
trigger:
  branches:
    include:
      - master

variables:
- group: Common

stages:

- template: templates/build.yml

- template: templates/deploy.yml
  parameters:
    environmentName: Development
    variableGroup: Dev
```

```
- template: templates/deploy.yml
  parameters:
    environmentName: QA
    variableGroup: QA
    dependsOn: Deploy_Development

- template: templates/deploy.yml
  parameters:
    environmentName: Production
    variableGroup: Prod
    dependsOn: Deploy_QA
```

Build template: templates/build.yml

```
stages:

- stage: Build
  displayName: 'Build and Test'

  jobs:

  - job: Build
    displayName: 'Build Solution'

    pool:
      vmImage: 'windows-latest'

    steps:

    - checkout: self
      clean: true

    - task: UseDotNet@2
      displayName: 'Install .NET SDK'
      inputs:
        packageType: 'sdk'
        version: '$(DotNetVersion)'

    - task: DotNetCoreCLI@2
      displayName: 'Restore NuGet Packages'
      inputs:
        command: 'restore'
        projects: '**/*.sln'

    - task: DotNetCoreCLI@2
      displayName: 'Build Solution'
      inputs:
        command: 'build'
        projects: '**/*.sln'
        arguments: '--configuration $(BuildConfiguration) --no-restore'

    - task: DotNetCoreCLI@2
      displayName: 'Run Unit Tests with Code Coverage'
      inputs:
        command: 'test'
        projects: '**/*UnitTests/*.csproj'
        publishTestResults: false
        arguments: >
          --configuration $(BuildConfiguration)
          --no-build
          --collect:"XPlat Code Coverage"
          --logger trx

    - task: PublishTestResults@2
      displayName: 'Publish Test Results'
      condition: succeededOrFailed()
      inputs:
        testResultsFormat: 'VSTest'
        testResultsFiles: '**/*.trx'
        failTaskOnFailedTests: true
        testRunTitle: 'Unit Tests'

    - task: PublishCodeCoverageResults@2
      displayName: 'Publish Code Coverage'
      condition: succeededOrFailed()
      inputs:
        codeCoverageTool: 'Cobertura'
        summaryFileLocation: '$(Agent.TempDirectory)**/coverage.cobertura.xml'
```

```

- task: DotNetCoreCLI@2
  displayName: 'Publish Application'
  inputs:
    command: 'publish'
    publishWebProjects: true
    arguments: '--configuration $(BuildConfiguration) --output $(Build.ArtifactStagingDirectory)'
    zipAfterPublish: true

- task: PublishBuildArtifacts@1
  displayName: 'Publish Artifact'
  inputs:
    pathToPublish: '$(Build.ArtifactStagingDirectory)'
    artifactName: 'drop'

```

Deployment template: templates/deploy.yml

```

parameters:
- name: environmentName
  type: string

- name: variableGroup
  type: string

- name: dependsOn
  type: string
  default: Build

stages:
- stage: Deploy_${{ parameters.environmentName }}

  displayName: 'Deploy to ${{ parameters.environmentName }}'

  dependsOn: ${{ parameters.dependsOn }}

  variables:
  - group: ${{ parameters.variableGroup }}

  jobs:
  - deployment: DeployWebApi

    displayName: 'Deploy Web API'

    environment: ${{ parameters.environmentName }}

    strategy:
      runOnce:
        deploy:
          steps:
            - download: current
              artifact: drop

            - task: AzureWebApp@1
              displayName: 'Deploy Azure Web App'
              inputs:
                azureSubscription: '$(AzureServiceConnection)'
                appType: 'webApp'
                appName: '$(AppServiceName)'
                package: '$(Pipeline.Workspace)/drop/**/*.*.zip'

```

Note: The exact task versions and input names can change as Azure DevOps task extensions evolve. The architectural concepts remain the same.

18. SonarCloud and Code Coverage

What coverage means

Code coverage measures how much of the executable code is exercised by tests. It is a measurement, not proof that the code is correct.

Coverage generation

```
dotnet test
↓
XPlat Code Coverage / Coverlet
↓
coverage.cobertura.xml
■■■ Azure DevOps coverage report
■■■ SonarCloud analysis
```

Coverage threshold

Publishing coverage does not itself enforce a minimum percentage. A threshold can be enforced through a SonarCloud Quality Gate, for example **New Code Coverage** $\geq 80\%$. Teams often prefer a New Code rule for legacy systems so old low-coverage code does not make every new PR fail.

SonarCloud analysis order

```
Prepare → Build → Test + Coverage → Analyze → Publish Quality Gate
```

Automatic vs CI-based analysis

Do not run SonarCloud Automatic Analysis and CI-based analysis simultaneously for the same project. In this course, Azure Pipelines performs CI-based analysis because we want explicit control over the analysis inside the pipeline.

The practical setup issue we encountered

The SonarCloud project was initially bound to an Azure DevOps repository even though the source/PR was in GitHub. That caused the PR lookup to fail. For GitHub PR analysis/decoration, the SonarCloud project needs the correct GitHub repository binding and integration.

19. Complete PR Validation YAML

Recommended file name: **pr-validation.yml**, at the repository root.

Complete file

```
trigger: none

pr:
  branches:
    include:
      - master

pool:
  vmImage: 'windows-latest'

variables:
- group: Common

steps:
- checkout: self
  clean: true

- task: UseDotNet@2
  displayName: 'Install .NET SDK'
  inputs:
    packageType: 'sdk'
    version: '$(DotNetVersion)'

- task: SonarCloudPrepare@3
  displayName: 'Prepare SonarCloud Analysis'
```

```

inputs:
  SonarCloud: 'SonarCloud-Connection'
  organization: 'YOUR_SONARCLOUD_ORGANIZATION_KEY'
  scannerMode: 'dotnet'
  projectKey: 'YOUR_SONARCLOUD_PROJECT_KEY'
  projectName: 'EmployeeManagementAPI'

- task: DotNetCoreCLI@2
  displayName: 'Restore NuGet Packages'
  inputs:
    command: 'restore'
    projects: '**/*.sln'

- task: DotNetCoreCLI@2
  displayName: 'Build Solution'
  inputs:
    command: 'build'
    projects: '**/*.sln'
    arguments: '--configuration $(BuildConfiguration) --no-restore'

- task: DotNetCoreCLI@2
  displayName: 'Run Unit Tests with Code Coverage'
  inputs:
    command: 'test'
    projects: '**/*UnitTests/*.csproj'
    publishTestResults: false
    arguments: >
      --configuration $(BuildConfiguration)
      --no-build
      --collect:"XPlat Code Coverage"
      --logger trx

- task: PublishTestResults@2
  displayName: 'Publish Test Results'
  condition: succeededOrFailed()
  inputs:
    testResultsFormat: 'VSTest'
    testResultsFiles: '**/*.trx'
    failTaskOnFailedTests: true
    testRunTitle: 'Unit Tests'

- task: PublishCodeCoverageResults@2
  displayName: 'Publish Code Coverage'
  condition: succeededOrFailed()
  inputs:
    codeCoverageTool: 'Cobertura'
    summaryFileLocation: '$(Agent.TempDirectory)**/coverage.cobertura.xml'

- task: SonarCloudAnalyze@3
  displayName: 'Run SonarCloud Analysis'

- task: SonarCloudPublish@3
  displayName: 'Publish SonarCloud Quality Gate'
  inputs:
    pollingTimeoutSec: '300'

```

What each Sonar setting means

- **SonarCloud:** the Azure DevOps service connection name.
- **organization:** SonarCloud organization key, not merely the display name.
- **projectKey:** unique SonarCloud project key.
- **scannerMode: dotnet:** use the .NET scanner.
- **pollingTimeoutSec: 300:** wait up to 300 seconds for the Quality Gate result.

Why succeededOrFailed() is used

Test-result and coverage publishing should still run when tests fail, so the developer gets diagnostic information. The pipeline remains failed if the tests failed.

20. GitHub + Azure DevOps Integration

Where PR validation is configured

The PR validation YAML lives in GitHub, but the pipeline itself is created and executed in Azure DevOps: Pipelines → New pipeline → GitHub → select repository → Existing Azure Pipelines YAML file → pr-validation.yml.

How the PR trigger works

```
GitHub PR event
↓
Azure DevOps receives the repository event
↓
Pipeline checks its `pr:` trigger
↓
Target branch matches master
↓
PR Validation pipeline starts
```

How merge triggers the main pipeline

```
PR validation → merge
↓
New commit on master
↓
Main pipeline `trigger` matches master
↓
Build → Artifact → Dev → QA → Prod
```

Branch protection

Because the source repository is GitHub, the rule preventing direct pushes to the protected branch is configured in GitHub. A required status check can be used to require the Azure Pipeline to pass before merge, subject to the GitHub plan/repository settings.

In the practical setup, the private repository showed 'Not enforced' under the available GitHub plan. That means the rule could be configured but would not actually prevent direct pushes until the repository/account is on a plan that supports enforcement.

21. Pipeline Troubleshooting

Troubleshooting mindset

```
Don't guess → identify the first failing step → read the first meaningful error
↓
Separate application errors from pipeline/integration errors
```

Failure: restore fails

- Check SDK/runtime version.
- Check NuGet feeds and authentication.

- Check solution/project glob patterns.
- Compare local SDK with pipeline SDK.

Failure: tests fail

- Open the test result summary.
- Inspect the first failing test.
- Check environment variables and external dependencies.
- Remember: `succeededOrFailed()` only controls reporting tasks; it does not make failed tests pass.

Failure: coverage report not found

- Verify that test execution generated coverage.
- Verify the expected Cobertura file exists.
- Check the `summaryFileLocation` glob.
- Ensure the coverage format matches the publishing task.

Failure: Sonar Prepare cannot download quality profile

Check the SonarCloud service connection, organization key, project key and project access. This occurred during the practical setup before build analysis could proceed.

Failure: Sonar says Automatic Analysis is enabled

Disable Automatic Analysis when using CI-based analysis through Azure Pipelines.

Failure: Sonar cannot find pull request / HTTP 404

Check the SonarCloud project's repository binding and DevOps-platform integration. In the practical setup, the project was bound to Azure DevOps while the actual repository/PR was in GitHub. That mismatch explained the PR lookup failure.

Failure: deployment says unauthorized

- Verify the Azure service connection.
- Verify that its identity has permission on the target Azure resource.
- Check the app/resource names supplied by the variable group.

Failure: artifact not found

Remember that a deployment job may run on a different agent. Confirm that the build actually published the artifact and that the deployment job downloads the same artifact name.

22. Interview Questions and Answers

What is CI?

Automatically validate integrated code changes through build, tests and quality checks.

What is CD?

Continuously keep software deployable and/or automatically deploy it through environments.

What is an agent?

The machine that executes pipeline jobs and steps.

Stage vs job?

A stage is a logical phase; a job is an execution unit within a stage.

Why use artifacts?

To transfer and promote build outputs across jobs/stages and preserve the exact tested output.

Why Build Once, Deploy Many?

It ensures Dev, QA and Production receive the same artifact.

What is an environment?

A deployment target abstraction with deployment history and optional approvals/checks.

What is a service connection?

An authenticated connection/identity used by Azure DevOps to access external resources.

Do I need separate service connections for Dev/QA/Prod?

Not inherently. One can be reused if it has the required permissions; separate connections can be chosen for least privilege.

Parameters vs variables?

Parameters influence template/pipeline expansion; variables are runtime values.

Why variable groups?

Centralize shared environment configuration and secrets.

Why use deployment jobs?

They integrate deployments with environments, history and checks.

What does dependsOn do?

Controls execution dependency/order between stages or jobs.

Why succeededOrFailed() for test publishing?

To publish diagnostics even when tests fail.

Where is a coverage threshold enforced?

It can be enforced by a quality gate such as SonarCloud; the coverage publishing task itself does not enforce a threshold.

Why Prepare before Build in Sonar?

The scanner needs to integrate with the build and collect analysis information.

Why not run Automatic and CI-based Sonar analysis together?

They are two different analysis mechanisms; using both for the same project can cause conflicts/duplicate analysis.

Why did PR validation run automatically?

The pipeline's PR trigger matched the GitHub PR target branch.

Why did CI/CD run after PR validation failed?

If the PR was still merged, the resulting push to the target branch can trigger the main pipeline. A failed validation only blocks merging when it is enforced as a required check.

Environment vs deployment strategy?

Environment is where; strategy is how the rollout is performed.

23. Final Cheat Sheet

One-minute explanation

For a .NET application in GitHub, I use an Azure DevOps PR validation pipeline to restore, build, test, generate coverage and run SonarCloud analysis. Once the PR is approved and merged, the main Azure pipeline builds the official artifact once and promotes that same artifact through Development, QA and Production. Deployment jobs target Azure DevOps Environments, service connections provide authenticated Azure access, variable groups provide environment-specific configuration, and approvals/checks are configured on environments rather than hard-coded into deployment tasks.

YAML syntax essentials

```
object:
  property: value

list:
- item1
- item2

listOfObjects:
- name: First
  value: One

steps:
- task: SomeTask@1
  displayName: 'My task'
  inputs:
    setting: 'value'
```

A dash starts a list item. Indentation expresses hierarchy. `task` and `displayName` in a task step are sibling properties of the same list item. `vmImage` is a child property of the `pool` object.

Final architecture

```
GitHub feature branch
↓
Pull Request
↓
PR Validation
Build → Test → Coverage → Sonar → Quality Gate
↓
Merge to master
↓
Main CI/CD
Build → Test → Publish Artifact
↓
Development → QA → Production
```

Most important principles

- WHY before HOW.
- Build once, deploy many.
- Keep configuration separate from code.
- Use environments for deployment targets and checks.
- Use service connections for authenticated access.
- Use templates to remove YAML duplication.
- Use PR validation to protect the quality of changes before merge.
- Use coverage as a measurement and a Quality Gate when a threshold is required.
- Troubleshoot from the first meaningful failure, not from the final cascade of errors.

End of Course Notes

This document consolidates the Azure DevOps Pipelines course covered in the conversation and stops at the requested point. It is intentionally optimized for a .NET developer interview rather than a dedicated DevOps engineer role.